1) Putting the ESP32 into Sleep Mode

When you run an ESP32 on battery power, you really can't leave it fully on all the time. It will drain the battery super fast. To fix this, we use something called Deep Sleep. It basically shuts down the main processor.But before you put it to sleep, you have to tell it how to wake up later. We usually use the internal timer for this. Once the timer finishes, the board wakes up and restarts from the setup() function.

Code Example:

```
// Conversion factor for micro-seconds
void setup() {
  Serial.begin(115200);
  Serial.println("Booting up.");
  // Set the timer alarm
  esp_sleep_enable_timer_wakeup(SECONDS_TO_SLEEP * uS_TO_S);
  Serial.println("Going to sleep to save power.");
  // Actually trigger the sleep mode
  esp_deep_sleep_start();
}
void loop() {
  // It never reaches this part because deep sleep restarts the board
}
```

2) How to set up I2C in Tinkercad

I2C is really handy because you only need two data wires to make two Arduinos talk to each other. One board acts as the boss (Master) and the other just listens (Slave).

In this setup, the master sends a '1' or a '0' every second. The slave board listens on address 8, and when it gets a '1', it turns its LED on. Otherwise, it turns it off.

```
#include <Wire.h>
void setup() {
  Wire.begin();
}
void loop() {
  Wire.beginTransmission(8); // Talk to device #8
  Wire.write(1); // Send a 1 to turn LED on
```

```
  Wire.endTransmission();
  delay(1000);
  Wire.beginTransmission(8);
  Wire.write(0); // Send a 0 to turn LED off
  Wire.endTransmission();
  delay(1000);
}
#include <Wire.h>
void setup() {
  Wire.begin(); // Join bus as master
}
void loop() {
  Wire.beginTransmission(8); // Talk to device #8
  Wire.write(1); // Send a 1 to turn LED on
  Wire.endTransmission();
  delay(1000);
  Wire.beginTransmission(8);
  Wire.write(0); // Send a 0 to turn LED off
  Wire.endTransmission();
  delay(1000);
}
```

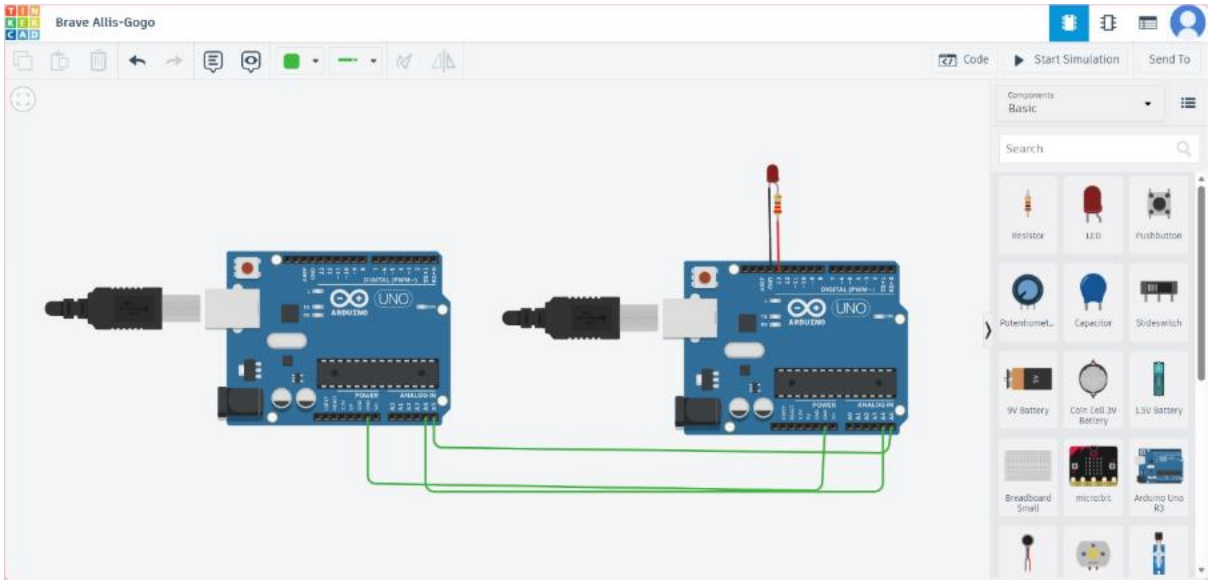
Master board

```
#include <Wire.h>
int myLed = 13;
void setup() {
  pinMode(myLed, OUTPUT);
  Wire.begin(8); // Assign ID 8 to this board
  Wire.onReceive(getData); // Jump to getData function when message arrives
}
void loop() {
}
```

```
void getData(int bytes) {
  int val = Wire.read();
  if (val == 1) {
    digitalWrite(myLed, HIGH);
  } else {
    digitalWrite(myLed, LOW);
  }
}
```

```
 #include <Wire.h>
int myLed = 13;
void setup() {
  pinMode(myLed, OUTPUT);
  Wire.begin(8); // Assign ID 8 to this board
  Wire.onReceive(getData); // Jump to getData function when message arrives
}
void loop() {
  // Nothing to do here
}
void getData(int bytes) {
  int val = Wire.read();
  if (val == 1) {
    digitalWrite(myLed, HIGH);
  } else {
    digitalWrite(myLed, LOW);
  }
}
```

Slave board



3) Basic SPI Setup

SPI is another communication method, but it's way faster than I2C. The master board uses a Chip Select (CS) pin. When we pull this pin LOW, it tells the slave device to get ready to receive data.

```
#include <SPI.h>

int chipSelect = 10;

void setup() {

  SPI.begin();

  pinMode(chipSelect, OUTPUT);

  Serial.begin(9600);

  Serial.println("Starting SPI");

}

void loop() {

  // Wake up the slave

  digitalWrite(chipSelect, LOW);

  // Send a character

  SPI.transfer('A');

  // Put slave back to sleep

  digitalWrite(chipSelect, HIGH);

  delay(1000);

}
```

4) UART Communication between Two Boards

This is the classic serial method. You connect the TX of one Arduino to the RX of the other.

In this project, the first board reads a temperature sensor and sends the calculated Celsius value over serial. The second board reads that incoming number and lights up different LEDs depending on if it's hot, cold, or normal.

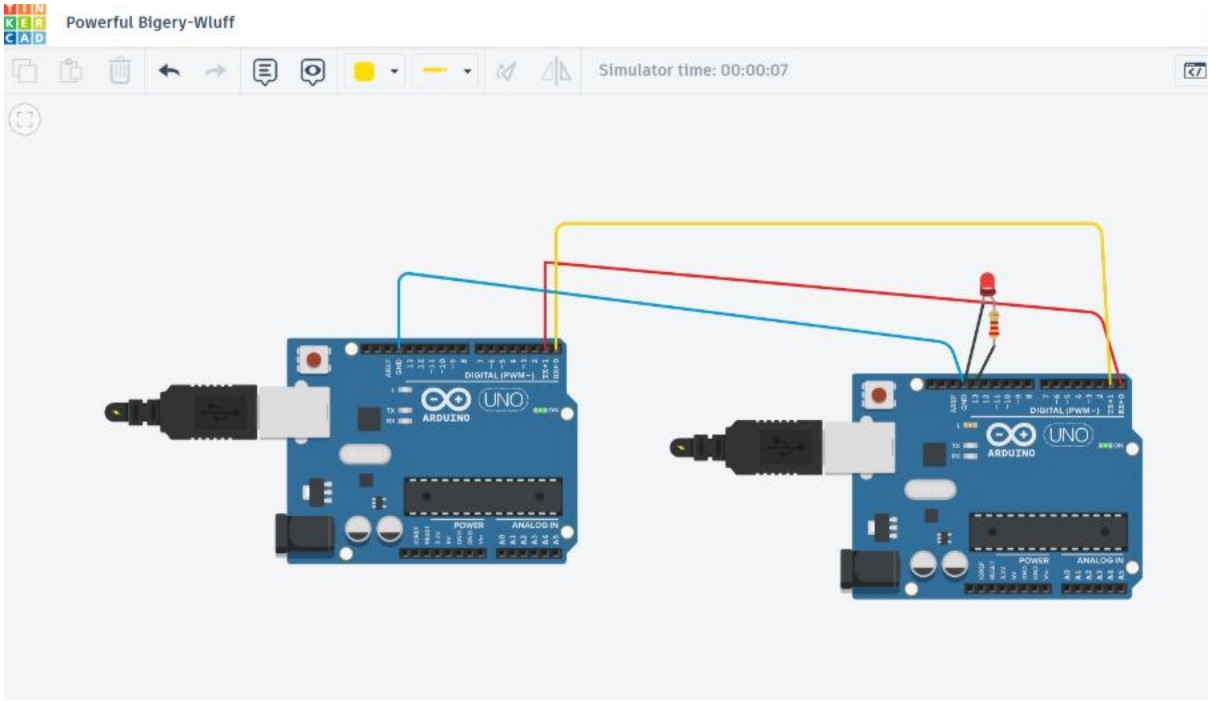
Transmitter

```
int tempSensor = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(tempSensor);
  // Doing the math to get Celsius
  float voltage = raw * (5.0 / 1023.0);
  float tempC = (voltage - 0.5) * 100;
  // Send it over TX
  Serial.println(tempC);
  delay(1000);
}
```

Reciever

```
float reading;
int coldPin = 8;
int normalPin = 9;
int hotPin = 10;
void setup() {
  Serial.begin(9600);
  pinMode(coldPin, OUTPUT);
  pinMode(normalPin, OUTPUT);
  pinMode(hotPin, OUTPUT);
}
void loop() {
  if (Serial.available()) {
    reading = Serial.parseFloat(); // Get the number
```

```
  if (reading < 25) {
    digitalWrite(coldPin, HIGH);
    digitalWrite(normalPin, LOW);
    digitalWrite(hotPin, LOW);
  }
  else if (reading < 35) {
    digitalWrite(coldPin, LOW);
    digitalWrite(normalPin, HIGH);
    digitalWrite(hotPin, LOW);
  }
  else {
    digitalWrite(coldPin, LOW);
    digitalWrite(normalPin, LOW);
    digitalWrite(hotPin, HIGH);
}
 }
}
```



5) Using All Arduino UNO Pins

To show how to use every single pin, I built a smart gate controller code. It takes inputs from analog sensors (for light, temp, and fuel) and digital inputs from a motion sensor and two buttons. Then it drives a bunch of LEDs, a buzzer for alarms, and a servo motor to physically open a gate.

```
#include <Servo.h>

Servo myGate;

// All our inputs

int tSensor = A0;

int fSensor = A1;

int lightSensor = A2;

int motionSensor = 2;

int lockBtn = 3;

int unlockBtn = 4;

// All our outputs

int blueLed = 5;

int greenLed = 6;

int redLed = 7;

int whiteLed = 8;

int warningLed = 10;

int speaker = 11;

void setup() {

  pinMode(blueLed, OUTPUT);

  pinMode(greenLed, OUTPUT);

  pinMode(redLed, OUTPUT);

  pinMode(whiteLed, OUTPUT);

  pinMode(warningLed, OUTPUT);

  pinMode(speaker, OUTPUT);

  // Using pullups so we don't need external resistors for buttons

  pinMode(motionSensor, INPUT);

  pinMode(lockBtn, INPUT_PULLUP);

  pinMode(unlockBtn, INPUT_PULLUP);

  digitalWrite(lockBtn, HIGH);

  digitalWrite(unlockBtn, HIGH);
```

```
 myGate.attach(9);
 myGate.write(0); // Gate closed by default
 Serial.begin(9600);
}
void loop() {
 // Handle Temperature
 int tVal = analogRead(tSensor);
 float volts = tVal * (5.0 / 1023.0);
 float actualTemp = (volts - 0.5) * 100;
 if (actualTemp < 25) {
  digitalWrite(blueLed, HIGH); digitalWrite(greenLed, LOW); digitalWrite(redLed, LOW);
 } else if (actualTemp < 35) {
  digitalWrite(blueLed, LOW); digitalWrite(greenLed, HIGH); digitalWrite(redLed, LOW);
 } else {
  digitalWrite(blueLed, LOW); digitalWrite(greenLed, LOW); digitalWrite(redLed, HIGH);
 }
  // Handle Light level
 int lVal = analogRead(lightSensor);
 if (lVal < 500) {
  digitalWrite(whiteLed, HIGH); // Turn on lights if it's getting dark
 } else {
  digitalWrite(whiteLed, LOW);
 }
 // Handle Fuel reading
 int fVal = analogRead(fSensor);
 if (fVal < 300) {
  digitalWrite(warningLed, HIGH);
 } else {
  digitalWrite(warningLed, LOW);
 }
 // Motion and Alarm
```

```
 if (digitalRead(motionSensor) == HIGH) {
   tone(speaker, 1000);
 } else {
   noTone(speaker);
 }
 // Button logic for the gate
 if (digitalRead(lockBtn) == LOW) {
   myGate.write(0);
   Serial.println("Gate is locked");
   delay(300); // Stop button bouncing
 }
 if (digitalRead(unlockBtn) == LOW) {
   myGate.write(90); // Open it up
   Serial.println("Gate is unlocked");
   delay(300);
 }
 // Print everything so we can debug
 Serial.print("Temp: "); Serial.print(actualTemp);
 Serial.print(" | Fuel: "); Serial.print(fVal);
 Serial.print(" | Light: "); Serial.println(lVal);
 delay(200);
}
```

By: Vidish Gupta